AKDENİZ UNİVERSİTY • COMPUTER ENGİNEERİNG
Advanced Web Programming • 2026

FinTreX

LangGraph Implementation Report

Homework 4 — LangGraph AI Agent Integration

Student	Egemen ÇAMÖZÜ (20220808076)
Submitted to	Melih Günay
Document Rev.	v1.0 — June 2026

"Bridging individual investors with professional financial consultants through a data-driven, AI-augmented consultation ecosystem."

Contents

1 Introduction **2**
1.1 Context and Previous Homeworks . . . 2
1.2 Project Repository . . . 2

2 System Architecture **3**

3 LangGraph Implementation (ai-service) **4**
3.1 Graph Architecture . . . 4
3.2 State Definition (`state.py`) . . . 4
3.3 Nodes . . . 5
3.3.1 Router Node (`router.py`) . . . 5
3.3.2 Tool Executor Node (`tool_executor.py`) . . . 5
3.3.3 Synthesizer Node (`synthesizer.py`) . . . 5
3.3.4 Guardrail Node (`guardrail.py`) . . . 6
3.4 Graph Compilation (`graph.py`) . . . 6
3.5 Tools Summary . . . 7
3.6 FastAPI Endpoints . . . 8

4 .NET Backend Integration **8**
4.1 New Entities . . . 8
4.2 Controller and Service . . . 9

5 Angular Frontend — AI Chat Page **9**
5.1 Key Components . . . 10
5.2 SSE Streaming with Fetch API . . . 10

6 Working Application Screenshots **11**

7 Coexistence, Security, and Conclusion **12**
7.1 CrewAI & LangGraph Coexistence . . . 12
7.2 Security Architecture . . . 13
7.3 Future Enhancements . . . 13
7.4 Conclusion . . . 13

1 Introduction

1.1 Context and Previous Homeworks

This report is a **direct continuation of Homework 3**, in which the **CrewAI**-based Pre-Analysis Assistant (PAA) was implemented. In HW4, a second AI system — the **LangGraph AI Portfolio Assistant** — is added to the platform. Both systems coexist within the same `ai-service/` microservice, serving different purposes for different audiences.

Table 1. CrewAI (HW3) vs. LangGraph (HW4) Comparison

Aspect	CrewAI (HW3)	LangGraph (HW4)
Purpose	Pre-analysis report for Economist	Interactive chat assistant for Users
Trigger	Task creation event (automatic)	User sends a message (on-demand)
Interaction	Fire-and-forget, single output	Multi-turn conversation with memory
Target User	Economist	User & Economist
Architecture	Sequential agent pipeline	Stateful graph with conditional edges
Endpoint	`POST /analyze`	`POST /ai-chat` + `/ai-chat/stream`
Streaming	No	Yes (SSE)
Safety	Prompt + regex validator	Prompt + guardrail node (feedback loop)

1.2 Project Repository

Git Repository

URL: https://github.com/egemencamozu/FinTreX

Updated Project Structure (HW4 additions highlighted)

```
FinTreX/
|-- ai-service/
|   |-- main.py                    # MODIFIED: +2 new endpoints
|   |-- config.py                  # MODIFIED: +LangGraph config
|   |-- requirements.txt           # MODIFIED: +langgraph deps
|   |-- crew.py, agents.py, ...    # UNCHANGED (CrewAI - HW3)
|   +-- langgraph_assistant/       # NEW MODULE
|       |-- state.py               # Graph state definition
|       |-- graph.py               # StateGraph + compile
|       |-- models.py              # Pydantic models
|       |-- concurrency.py         # Conversation lock
|       |-- http_client.py         # Backend API helper
```

```
|        |-- nodes/
|        |    |-- router.py          # Intent classification
|        |    |-- tool_executor.py   # Tool invocation
|        |    |-- synthesizer.py     # Response generation
|        |    +-- guardrail.py       # Advice detection
|        |-- tools/
|        |    |-- portfolio_tools.py
|        |    |-- market_data_tools.py
|        |    +-- calculation_tools.py
|        +-- prompts/
|             |-- system_prompt.py
|             +-- router_prompt.py
|-- backend/                         # .NET 8 (+AI Assistant layer)
|-- frontend/                        # Angular (+AI Chat feature)
+-- mobile/                          # Flutter
```

2 System Architecture

The LangGraph AI Assistant operates as a **new module** within the existing `ai-service` Python microservice. It communicates with the .NET backend through REST APIs and exposes SSE streaming for real-time responses.

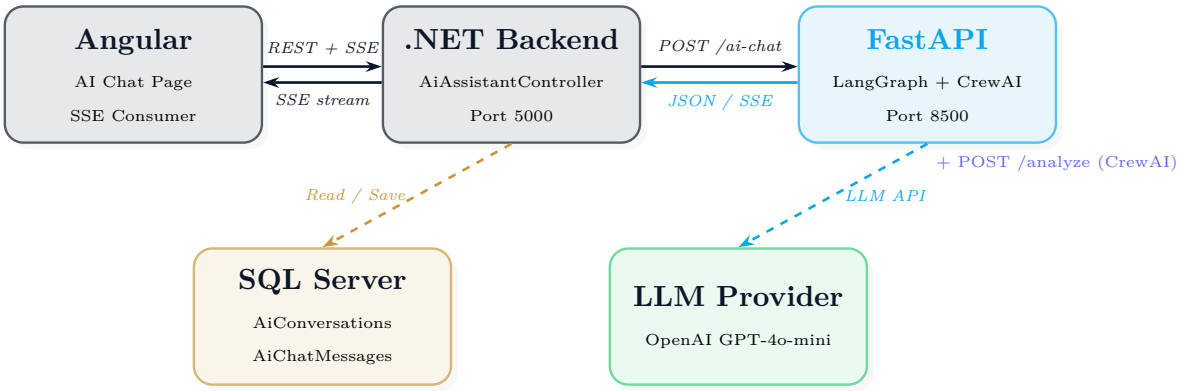

Figure 1. Full-Stack Architecture — LangGraph alongside CrewAI

3 LangGraph Implementation (ai-service)

3.1 Graph Architecture

The LangGraph assistant uses a **stateful directed graph** with four nodes and conditional edges, including a **guardrail feedback loop** for safety enforcement.

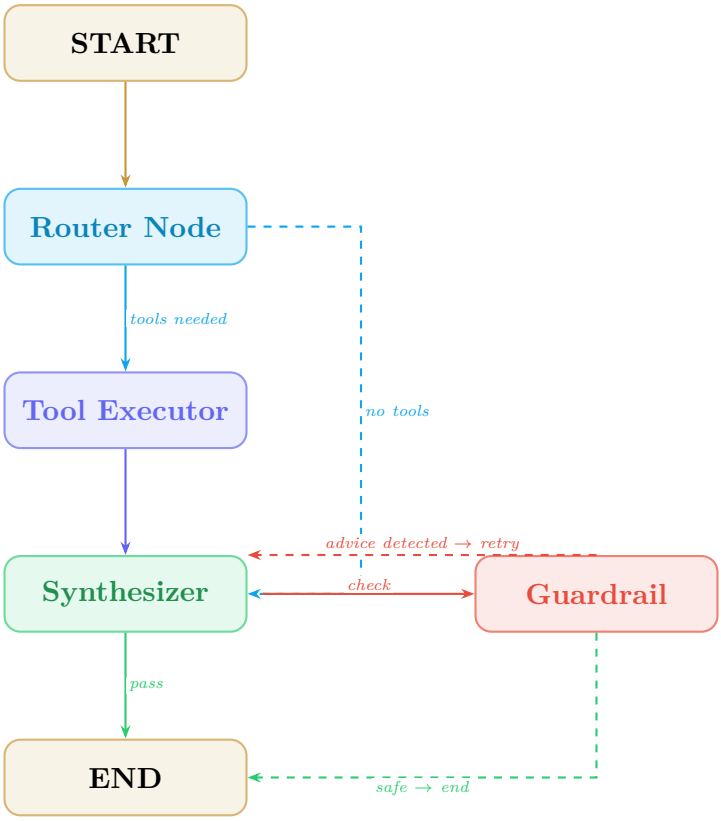

Figure 2. LangGraph StateGraph — Node & Edge Architecture

3.2 State Definition (`state.py`)

```python
from typing import TypedDict, Annotated, Optional
from operator import add

class AssistantState(TypedDict):
    # Input
    user_message: str
    user_id: str
    user_role: str            # "USER" | "ECONOMIST"
    auth_token: str
    conversation_history: list[dict]

    # Router output
    selected_tools: list[str]
    requires_tools: bool

    # Tool results
    tool_results: Annotated[list[dict], add]
    tool_errors: Annotated[list[str], add]
    partial_data: bool

    # Synthesizer output
    response_text: str

    # Guardrail
    guardrail_passed: bool
```

```
    guardrail_retry_count: int
```

Listing 1. state.py — LangGraph State (TypedDict)

3.3 Nodes

3.3.1 Router Node (`router.py`)

The router uses an LLM call to analyze the user's message and determine which tools are needed. It outputs a list of tool names and a boolean flag.

```
async def router_node(state: AssistantState) -> dict:
    llm = get_langgraph_llm()
    prompt = build_router_prompt(
        state["user_message"],
        state["conversation_history"]
    )
    response = await llm.ainvoke(prompt)
    parsed = parse_router_response(response.content)

    return {
        "selected_tools": parsed["tools"],
        "requires_tools": len(parsed["tools"]) > 0,
    }
```

Listing 2. router.py — Intent classification and tool selection

3.3.2 Tool Executor Node (`tool_executor.py`)

Executes selected tools by calling the .NET backend API through an HTTP client. Handles failures gracefully — partial results are passed to the synthesizer.

```
TOOL_REGISTRY = {
    "get_user_portfolios": get_user_portfolios,
    "get_portfolio_detail": get_portfolio_detail,
    "get_stock_prices": get_stock_prices,
    "get_crypto_prices": get_crypto_prices,
    "get_gold_prices": get_gold_prices,
    "get_forex_rates": get_forex_rates,
    "calculate_portfolio_distribution": calc_distribution,
    "calculate_total_value": calc_total_value,
    # ... additional tools
}

async def tool_executor_node(state: AssistantState) -> dict:
    results, errors = [], []
    for tool_name in state["selected_tools"]:
        fn = TOOL_REGISTRY.get(tool_name)
        if fn is None:
            errors.append(f"Unknown tool: {tool_name}")
            continue
        try:
            result = await fn(state)
            results.append({"tool": tool_name, "data": result})
        except Exception as e:
            errors.append(f"{tool_name}: {str(e)}")

    return {
        "tool_results": results,
        "tool_errors": errors,
        "partial_data": len(errors) > 0 and len(results) > 0,
    }
```

Listing 3. tool_executor.py — Tool invocation with error handling

3.3.3 Synthesizer Node (`synthesizer.py`)

Converts tool results into a natural-language Turkish response using the system prompt.

```python
async def synthesizer_node(state: AssistantState) -> dict:
    llm = get_langgraph_llm()
    prompt = build_synthesis_prompt(
        user_message=state["user_message"],
        tool_results=state["tool_results"],
        tool_errors=state["tool_errors"],
        conversation_history=state["conversation_history"],
    )
    response = await llm.ainvoke(prompt)
    return {"response_text": response.content}
```

Listing 4. synthesizer.py — Natural language response generation

3.3.4 Guardrail Node (guardrail.py)

The guardrail inspects the synthesizer's output for investment advice. If prohibited content is detected, it triggers a **feedback loop** back to the synthesizer with a corrective instruction.

```python
PROHIBITED_PATTERNS = [
    r"\b(tavsiye|oneri|oneriyorum)\b",
    r"\b(almali|satmali|tutmali)\b",
    r"\b(satin al|sat|tut)\b",
    r"\b(should buy|should sell|recommend)\b",
]

async def guardrail_node(state: AssistantState) -> dict:
    text = state["response_text"].lower()
    for pattern in PROHIBITED_PATTERNS:
        if re.search(pattern, text):
            retry = state.get("guardrail_retry_count", 0)
            if retry >= GUARDRAIL_MAX_RETRY:
                return {
                    "response_text": "Yatirim tavsiyesi veremem. "
                        "Sadece portfoy verilerinizi "
                        "objektif olarak sunabilirim.",
                    "guardrail_passed": True,
                }
            return {
                "guardrail_passed": False,
                "guardrail_retry_count": retry + 1,
            }
    return {"guardrail_passed": True}
```

Listing 5. guardrail.py — Investment advice detection with retry loop

3.4 Graph Compilation (graph.py)

```python
from langgraph.graph import StateGraph, END
from .state import AssistantState

def build_graph():
    graph = StateGraph(AssistantState)

    # Add nodes
    graph.add_node("router", router_node)
    graph.add_node("tool_executor", tool_executor_node)
    graph.add_node("synthesizer", synthesizer_node)
    graph.add_node("guardrail", guardrail_node)

    # Entry
    graph.set_entry_point("router")

    # Conditional: router -> tools or synthesizer
    graph.add_conditional_edges("router", route_decision, {
        "tools": "tool_executor",
        "direct": "synthesizer",
    })

```

```python
    # tool_executor -> synthesizer
    graph.add_edge("tool_executor", "synthesizer")

    # synthesizer -> guardrail
    graph.add_edge("synthesizer", "guardrail")

    # Conditional: guardrail -> end or retry
    graph.add_conditional_edges("guardrail", guard_decision, {
        "safe": END,
        "retry": "synthesizer",
    })

    return graph.compile()
```

Listing 6. graph.py — StateGraph definition with conditional edges

LangGraph Key Features Used

- **StateGraph** with `TypedDict` — type-safe state management
- **Conditional Edges** — dynamic routing based on router and guardrail output
- **Feedback Loop** — guardrail → synthesizer retry cycle (max 2 retries)
- **Tool Use** — 11 tools calling .NET backend via HTTP
- **Annotated Reducers** — `Annotated[list, add]` for accumulating tool results

3.5 Tools Summary

Table 2. LangGraph Tool Registry

Tool Name	Category	Description
`get_user_portfolios`	Portfolio	List all portfolios for the user
`get_portfolio_detail`	Portfolio	Assets and values for a specific portfolio
`get_client_portfolios`	Portfolio	Economist: view client's portfolios
`get_stock_prices`	Market	BIST stock prices
`get_stock_detail`	Market	Detailed info for a single stock
`get_crypto_prices`	Market	Cryptocurrency prices
`get_gold_prices`	Market	Precious metal prices
`get_forex_rates`	Market	USD/TRY, EUR/TRY exchange rates
`calc_distribution`	Calculation	Asset type distribution (percentages)
`calc_total_value`	Calculation	Portfolio total in TRY and USD
`calc_pnl`	Calculation	Profit/Loss calculation

3.6 FastAPI Endpoints

Two new endpoints were added to the existing `main.py`, alongside the CrewAI `/analyze` endpoint:

```python
from langgraph_assistant.models import AiChatRequest, AiChatResponse

@app.post("/ai-chat", response_model=AiChatResponse)
async def ai_chat(request: AiChatRequest):
    """Standard (non-streaming) AI chat endpoint."""
    graph = build_graph()
    result = await graph.ainvoke({...})
    return AiChatResponse(
        conversation_id=request.conversation_id or str(uuid4()),
        message=result["response_text"],
        tools_used=[r["tool"] for r in result["tool_results"]],
        partial_data=result.get("partial_data", False),
        is_successful=True,
    )

@app.post("/ai-chat/stream")
async def ai_chat_stream(request: AiChatRequest):
    """SSE streaming endpoint --- tokens flow in real-time."""
    async def event_generator():
        # ... yields SSE events: tool_start, token, done
        pass
    return EventSourceResponse(event_generator())
```

Listing 7. main.py — New LangGraph endpoints (excerpt)

4 .NET Backend Integration

4.1 New Entities

Two new entity classes were added, **completely independent** from the existing `Conversation`/`ChatMessage` entities (which serve user–economist chat):

```csharp
public class AiConversation {
    public int Id { get; set; }
    public string UserId { get; set; }
    public string? Title { get; set; }
    public DateTime CreatedAtUtc { get; set; }
    public DateTime? LastMessageAtUtc { get; set; }
    public bool IsActive { get; set; } = true;
    public bool IsProcessing { get; set; } = false;
    public ICollection<AiChatMessage> Messages { get; set; }
}

public class AiChatMessage {
    public long Id { get; set; }
    public int AiConversationId { get; set; }
    public AiMessageRole Role { get; set; }    // User | Assistant
    public string Content { get; set; }
    public string? ToolsUsed { get; set; }     // JSON array
    public bool PartialData { get; set; }
    public DateTime SentAtUtc { get; set; }
}
```

Listing 8. AiConversation.cs and AiChatMessage.cs (key fields)

4.2 Controller and Service

```
[ApiController]
[Route("api/v1/[controller]")]
[Authorize(Roles = "User,Economist")]
public class AiAssistantController : ControllerBase
{
    [HttpPost("chat")]
    public async Task<ActionResult<AiChatResponseDto>>
        SendMessage(AiChatRequestDto request) { ... }

    [HttpPost("chat/stream")]
    public async Task StreamMessage(AiChatRequestDto request)
    { /* SSE proxy to ai-service */ }

    [HttpGet("conversations")]
    public async Task<ActionResult<List<AiConversationListDto>>>
        GetConversations() { ... }

    [HttpGet("conversations/{id}")]
    public async Task<ActionResult<AiConversationDto>>
        GetConversation(int id) { ... }

    [HttpDelete("conversations/{id}")]
    public async Task<IActionResult>
        DeleteConversation(int id) { ... }
}
```

Listing 9. AiAssistantController.cs — API endpoints

Design Decisions

- **JWT Forward:** User's token is forwarded to ai-service, which uses it to call backend APIs. No service-account tokens.
- **IsProcessing Lock:** Prevents concurrent requests to the same conversation (returns 429).
- **Soft Delete:** Conversations are hidden (`IsActive = false`), not destroyed.
- **Conversation History:** Last 20 messages are loaded from DB and injected into LangGraph state for multi-turn context.

5 Angular Frontend — AI Chat Page

A new feature module `ai-assistant/` was added to the Angular application with a two-panel layout: conversation list on the left, chat window on the right.

5.1 Key Components

Table 3. AI Assistant Angular Components

Component	Purpose
`ai-chat`	Page component — 2-panel layout (sidebar + chat)
`ai-conversation-list`	Left panel — list of past conversations
`ai-chat-window`	Right panel — message display + input
`ai-message-bubble`	Individual message (user or AI), with tool badges
`ai-typing-indicator`	Animated dots during AI processing
`ai-chat-input`	Text input with send button, disabled during processing
`ai-empty-state`	Welcome screen with suggested questions

5.2 SSE Streaming with Fetch API

Angular's `HttpClient` cannot consume POST-based SSE streams. The repository implementation uses the native `fetch()` API with `ReadableStream`:

```
streamMessage(request: AiChatRequest): Observable<string> {
  return new Observable(observer => {
    const token = this.auth.getToken() ?? '';
    fetch(`${this.baseUrl}/chat/stream`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${token}`,
      },
      body: JSON.stringify(request),
    }).then(async response => {
      const reader = response.body!
        .pipeThrough(new TextDecoderStream())
        .getReader();
      while (true) {
        const { done, value } = await reader.read();
        if (done) break;
        // Parse SSE lines: "data: {...}"
        for (const line of value.split('\n')) {
          if (line.startsWith('data: '))
            observer.next(line.slice(6));
        }
      }
      observer.complete();
    });
  });
}
```

Listing 10. ai-assistant-api.repository.ts — SSE streaming via fetch

6 Working Application Screenshots

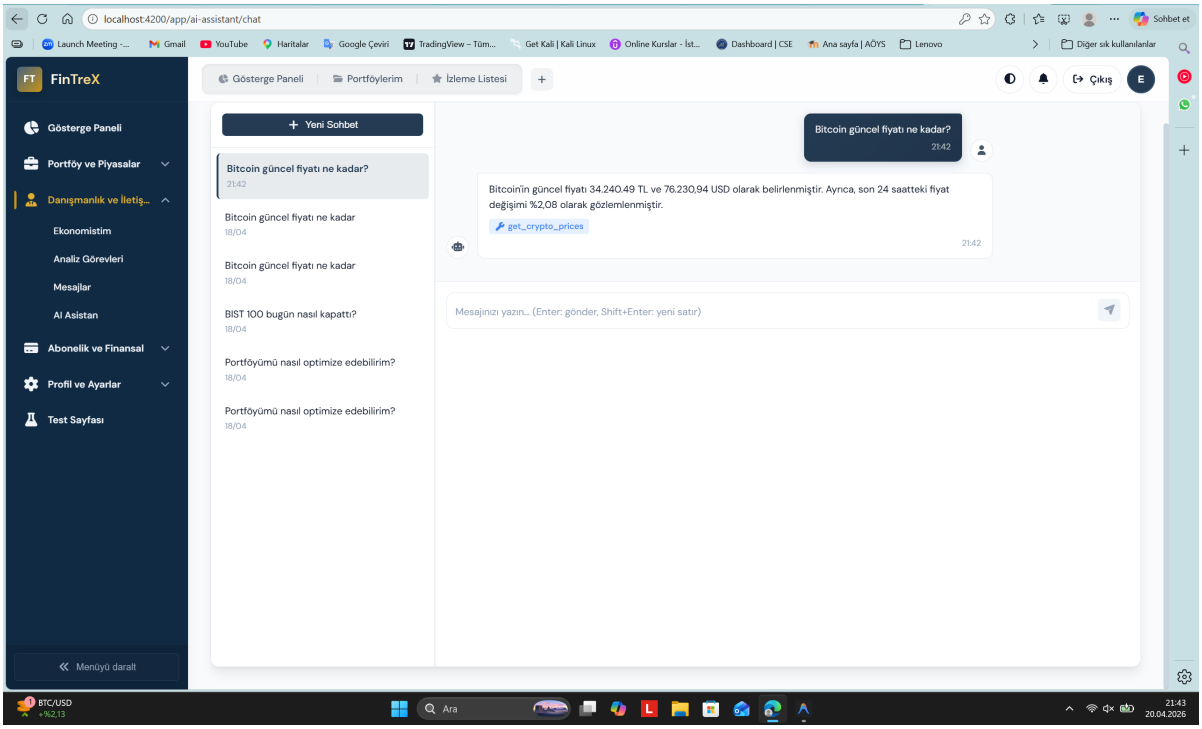

Figure 3. AI Portfolio Assistant — Chat Page. Two-panel layout showing the conversation list on the left and the active chat on the right. The user asks a portfolio question and receives an AI-generated, data-driven response with tool usage badges.

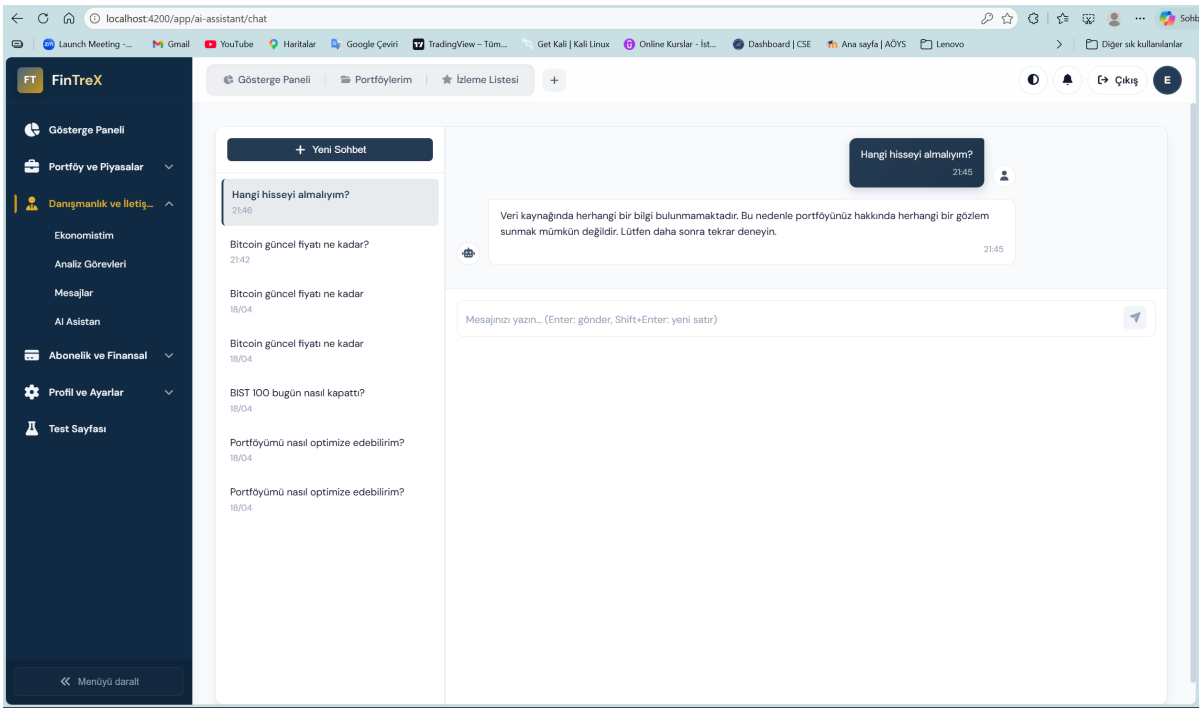

Figure 4. Guardrail — Investment Advice Rejection. When the user asks "Bitcoin al mıyım?" (Should I buy Bitcoin?), the guardrail node detects the advisory intent and returns a safe, factual response instead, enforcing the no-financial-advice policy.

Figure 5. Sidebar Integration. The "AI Asistan" menu item is visible for both User and Economist roles. Admin users do not see this item — access is enforced by the backend `[Authorize(Roles = "User,Economist")]` attribute.

7 Coexistence, Security, and Conclusion

7.1 CrewAI & LangGraph Coexistence

Both AI systems run within the same Python process and share the same LLM configuration, but they are **completely independent** in code, endpoints, and purpose:

Table 4. Side-by-Side Operation

Dimension	CrewAI (PAA)	LangGraph (Assistant)
Code Path	`agents.py, crew.py, tasks.py`	`langgraph_assistant/*`
Endpoint	`POST /analyze`	`POST /ai-chat[/stream]`
Trigger	.NET task creation	User message
State	Stateless per-request	Conversation history
Safety	Prompt + regex	Prompt + guardrail node

7.2 Security Architecture

Table 5. Security Measures — LangGraph Layer

Concern	Measure
Investment Advice	Guardrail node with regex + LLM fallback (max 2 retries)
Authentication	JWT forwarded from frontend → backend → ai-service
Authorization	`[Authorize(Roles = "User,Economist")]` on controller
Conversation Ownership	`GetByIdForUserAsync` filters by `UserId`
Concurrency	`IsProcessing` flag with timeout cleanup
Data Access	AI tools call backend GET endpoints only (read-only)
Input Validation	4000 char limit on messages (Pydantic + DTO)

7.3 Future Enhancements

Table 6. Post-MVP Roadmap

Feature	Description
LangSmith Integration	Trace and monitor LangGraph runs for debugging and optimization
Economist Client Selector	Allow Economists to query specific client portfolios via AI
Rate Limiting	Per-user request throttling to control LLM costs
Multi-LLM Failover	OpenAI → Claude → Azure fallback chain
Suggested Questions	Dynamic question chips based on portfolio composition

7.4 Conclusion

The LangGraph integration for FinTreX has been successfully implemented across three layers:

1. **AI-Service (Python):** Stateful graph with 4 nodes, 11 tools, conditional edges, guardrail feedback loop, and SSE streaming.
2. **Backend (.NET):** New entities, repository, service, and controller with JWT forwarding, conversation persistence, and concurrency control.

3. **Frontend (Angular):** Two-panel AI Chat page with SSE streaming via fetch API, optimistic UI, and conversation management.

LangGraph and CrewAI coexist in the same microservice without interference — each serves a distinct purpose within the FinTreX ecosystem. The guardrail feedback loop ensures that the no-financial-advice policy is enforced at every level, consistent with the platform's core design principle.